# BCR-6605 – Part F and Part G

**Part G is implemented** – one file to replace, test steps below.
**Part F is blocked** on information only Benoit has; the data side is proven with a query.

## Status

| Part | State | What exists |
|---|---|---|
| **F** – DACOM extraction | Blocked | A working extract query that returns every deviation field. The format, the column names and how DACOM receives it are unknown |
| **G** – automatic purge | **Done** | `PurgeDataManager` now deletes acknowledged deviations on the existing auto-purge schedule |

## Part G – the change

**File:** `PurgeDataManager.java` | **Package:** `com.eurocopter.mgs.service.purge.ejb` | **Module:** server

Replace the whole class with the file supplied alongside this document. Two changes, nothing else.

### 1. In `processPurge(...)`, one line added

```
        purgeFlights(aircraftId, getFlightToDeleteIds(aircraftId, maxDurationAllowed), progressCallback);
        // BCR-6605 purge acknowledged deviations on the same schedule and retention as the flights
        purgeAcknowledgedDeviations(maxDurationAllowed);
```

### 2. New method

```
   private void purgeAcknowledgedDeviations(long maxDurationAllowed) {
      long start = System.currentTimeMillis();
      LOGGER.info("Start purge deviation data operation ...");
      try {
         final Date limit = new Date(start - maxDurationAllowed);
         transaction.begin();
         int deleted = manager
               .createNativeQuery("DELETE FROM indicator_deviation WHERE end_date IS NOT NULL AND end_date < ?")
               .setParameter(1, limit)
               .executeUpdate();
         transaction.commit();
         manager.clear();
         LOGGER.info("End purge deviation data operation in {} ms, {} deviation(s) deleted.",
               (System.currentTimeMillis() - start), deleted);
      } catch (Exception e1) {
         LOGGER.error("Cannot purge deviations", e1);
         try {
            transaction.rollback();
         } catch (Exception e2) {
            LOGGER.error("Rollback problem", e2);
         }
      }
   }
```

### Why it is built this way

| Decision | Reason |
|---|---|
| `end_date IS NOT NULL` | The safety rule. An **active** deviation has a null end date and can never be deleted, however old. Only acknowledged ones are purged |
| Rides the existing auto purge | No second scheduler, no second setting. It runs inside `processPurge`, which the FLIGHT_AUTO_PURGE background operation already calls |
| Same retention as the flights | Uses the `maxDurationAllowed` already computed from `mgs.configuration.datastorage.autopurge` |
| Native query, own transaction | Matches the pattern the class already uses in `purgeFdcrDataBySize` – `transaction.begin()` / query / `commit()`, rollback on failure |
| Not scoped to one aircraft | The join from `indicator_configuration` up to the aircraft is not something I could verify. The delete is idempotent, so running it once per aircraft in a pass is harmless – the second run finds nothing. **Confirm with Benoit** |

## Part G – how to test

### Step 1 – create and acknowledge a deviation

Create one on the Health page, then tick it in the **Deviation list** tab and press Delete. Confirm it appears in **Deviation history**.

### Step 2 – age it past the retention window

```
SELECT id, te_reference, start_date, end_date
FROM public.indicator_deviation
WHERE end_date IS NOT NULL ORDER BY end_date DESC LIMIT 5;

UPDATE public.indicator_deviation
SET end_date = now() - interval '400 days'
WHERE id = '<paste the id>';
```

### Step 3 – create a control

Create a second deviation and **leave it active**. Do not acknowledge it. This proves the safety rule.

### Step 4 – run the purge

Trigger the auto purge the way you normally do – the scheduled run, or the purge action in the client under data storage.

### Step 5 – check the result

```
SELECT id, te_reference, start_date, end_date
FROM public.indicator_deviation ORDER BY start_date DESC;
```

| # | Row | Expected |
|---|---|---|
| a | Acknowledged and aged past retention | **Deleted** |
| b | Active, null `end_date` | **Still there** – this is the one that matters |
| c | Acknowledged but inside the retention window | Still there |

### Step 6 – confirm in the server log

```
End purge deviation data operation in N ms, 1 deviation(s) deleted.
```

If the count says 0 but the row disappeared, something else deleted it – report that rather than accepting the result.

## Part F – what exists today

The searches for `DACOM`, `extract`, `export` and `gsc` found nothing in the workspace, so there is no existing extract to copy. What can be proven is that every field DACOM could want is in the database and reachable in one query.

```
SELECT
   i.name                AS indicator,
   d.severity            AS severity,
   d.te_reference        AS te,
   d.start_date          AS valid_from,
   d.end_date            AS valid_to,
   d.threshold_level     AS level,
   d.threshold_type      AS type,
   d.threshold_value     AS value,
   d.min_value           AS lt_min,
   d.max_value           AS lt_max,
   d.max_points_per_hour AS points,
   fl.name               AS filtering_law
FROM public.indicator_deviation d
JOIN public.indicator_configuration ic ON ic.id = d.indicator_configuration_id
JOIN public.indicator i              ON i.id  = ic.indicator_id
LEFT JOIN public.filtering_law fl    ON fl.id = d.filtering_law_id
ORDER BY i.name, d.severity, d.start_date;
```

Add `WHERE d.end_date IS NULL` for active deviations only.

**Run it.** If the values match what you typed on the Health page, the data side of Part F is proven and only the format and the delivery remain. The aircraft column is missing because the join from `indicator_configuration` up to the aircraft is not yet known – the same gap as the per-aircraft purge.

## Questions for Benoit

One message covers both parts.

> **Part F – DACOM extraction**
>
> 1. What is DACOM, and how does it receive the data – a file we write, a database view it reads, or an interface we call?
> 2. If it is a file: which format (CSV, XML, fixed width), and which columns, in what order and under what names?
> 3. Active deviations only, or the history as well?
> 4. One extract per aircraft, or all aircraft together?
> 5. Produced on demand from a menu, or on a schedule?

> **Part G – automatic purge**
>
> 1. What retention period for deviations – the same as the flight auto purge (`mgs.configuration.datastorage.autopurge`), or its own setting?
> 2. Should the purge clear deviations for all aircraft in one pass, or only for the aircraft being purged?
> 3. Confirm that only acknowledged deviations are ever deleted, and an active deviation is kept whatever its age.

## What is left overall

| Item | State |
|---|---|
| Part G | **Implemented.** Test with the six steps above |
| Part F | Query proven. Format and delivery blocked on question 1 and 2 |
| Per-aircraft scoping | Affects both. Needs the `indicator_configuration` → aircraft join. Run `SELECT column_name FROM information_schema.columns WHERE table_name = 'indicator_configuration';` and send the result – that is enough to settle it |